ALGORITHM SPECIFICATION & ANALYSIS REPORT

Kosaraju's Algorithm: Strongly Connected Components

Linear-time two-pass depth-first traversal for directed graph component decomposition

1. Algorithm Code (Part I: Traversal Methods)

Below is Part 1 of the complete Java implementation, containing the DFS ordering traversal (Pass 1) and the transposed graph component extraction method (Pass 2):

```java
import java.util.*;

public class KosarajusAlgo {

    static void dfs1(
            int node,
            List<List<Integer>> graph,
            boolean[] visited,
            Stack<Integer> stack) {

        visited[node] = true;

        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                dfs1(neighbor, graph, visited, stack);
            }
        }

        stack.push(node);
    }

    static void dfs2(
            int node,
            List<List<Integer>> reversed,
            boolean[] visited) {

        visited[node] = true;
        System.out.print((char) ('A' + node) + " ");

        for (int neighbor : reversed.get(node)) {
            if (!visited[neighbor]) {
                dfs2(neighbor, reversed, visited);
            }
        }
    }
```

Core Dual Traversal: `dfs1` pushes vertices to the stack in post-order order of finishing times. `dfs2` runs on the transposed graph to extract one complete strongly connected component per iteration.

1. Algorithm Code (Part II: Graph Setup & Execution)

Below is Part 2 of the code, detailing directed graph construction, edge transposition, Pass 1 execution, and Pass 2 component output:

```
    public static void main(String[] args) {

        int n = 7;

        List<List<Integer>> graph = new ArrayList<>();
        List<List<Integer>> reversed = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
            reversed.add(new ArrayList<>());
        }

        graph.get(0).add(1); // A -> B
        graph.get(1).add(2); // B -> C
        graph.get(2).add(0); // C -> A
        graph.get(1).add(3); // B -> D
        graph.get(3).add(4); // D -> E
        graph.get(4).add(5); // E -> F
        graph.get(5).add(3); // F -> D
        graph.get(5).add(6); // F -> G
        graph.get(6).add(6); // G -> G

        for (int u = 0; u < n; u++) {
            for (int v : graph.get(u)) {
                reversed.get(v).add(u);
            }
        }

        boolean[] visited = new boolean[n];
        Stack<Integer> stack = new Stack<>();

        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                dfs1(i, graph, visited, stack);
            }
        }

        Arrays.fill(visited, false);

        int count = 0;

        while (!stack.isEmpty()) {
            int node = stack.pop();

            if (!visited[node]) {
                count++;
                System.out.print("SCC reversed " + count + ": ");
                dfs2(node, reversed, visited);
                System.out.println();
            }
        }
    }
}
```

2. Output & Output Explanation

```
SCC reversed 1: A C B SCC reversed 2: D F E SCC reversed 3: G
```

The program decomposes the 7-node directed graph into three maximal strongly connected components:

- **SCC 1 (Vertices {A, B, C}):** Form a closed directed cycle: $A \to B \to C \to A$. All three vertices are mutually reachable. Edge B → D exists, but no path exists from D back into {A, B, C}.
- **SCC 2 (Vertices {D, E, F}):** Form the second directed cycle: $D \to E \to F \to D$. While F connects to G, no directed edges return from G to this subgraph.
- **SCC 3 (Vertex {G}):** Possesses a self-loop $G \to G$. It has no outgoing edges to any other vertices, acting as an isolated terminal sink component.

3. Detailed Code Walkthrough

Code Component	Algorithmic Mechanism & Purpose
`dfs1(...)`	Traverses the forward graph. Pushes each node to `stack` after all descendant paths are explored, ordering vertices by completion time.
`reversed` adjacency list	Stores the graph transpose G^T. If an edge $u \to v$ exists in G, $v \to u$ is added to $reversed$.
`dfs2(...)`	Traverses the transposed graph starting from unvisited nodes popped from `stack`, extracting one complete SCC per invocation.
`Arrays.fill(visited, false)`	Resets the visited tracking flags before Pass 2 so vertices can be re-evaluated on the reversed graph.
Stack processing loop	Pops nodes in decreasing order of finish time, guaranteeing that search roots originate in source components of G (which become sinks in G^T).

4. Algorithmic Foundation & Working Principle

Kosaraju's algorithm relies on two fundamental theorems of directed graph theory:

- **Component DAG Inversion:** Contracting every SCC into a single super-vertex yields a Directed Acyclic Graph (DAG). Reversing the edges inverts the topological ordering without altering reachability inside any component.
- **Confinement Property:** In G^T, cross-component edges point from sink components to source components. Starting Pass 2 from the highest node on the stack isolates the search strictly within that component.

TIME COMPLEXITY	AUXILIARY SPACE	COMPONENT DAG
Θ(V + E)	**Θ(V + E)**	**Strictly Acyclic**
Two full DFS passes over vertices and edges.	Transposed adjacency list and tracking stack.	Condensing SCCs yields a valid topological order.

5. Practical Uses & Algorithm Utility

Key applications in software engineering and discrete mathematical optimization include:

- **2-SAT Solvers:** Tests boolean satisfiability by verifying that no literal x and its negation $\neg x$ lie within the same SCC.
- **Deadlock Resolution:** Isolates cyclic dependency cycles in wait-for graphs within database engines and operating system schedulers.
- **Graph Condensation:** Simplifies dense cyclic microservice call graphs into acyclic DAGs for dependency resolution.

6. Real-World Industry Use Cases

Industry Domain	Operational Problem	Kosaraju's Application
Social Networks (e.g., LinkedIn, X)	Directed engagement and follower networks.	Identifies tightly knit user communities with bidirectional influence paths for targeted recommendation.
Compiler Optimization (e.g., LLVM)	Control Flow Graph (CFG) analysis.	Detects strongly connected basic blocks representing nested programming loops for loop unrolling and optimization.
Web Search Engines	Hyperlink structure indexing and crawling.	Partitions the web graph into circulation clusters, avoiding crawler traps and optimizing PageRank computations.
Logistics & Routing	Circular delivery route validation.	Verifies that all urban distribution hubs maintain return paths, ensuring resilient delivery logistics.